

CS/CE 224/227: Object Oriented Programming and Design Methodologies

Week 01/Unit 01: Introduction

Course Taught by: Zubair Irshad

Course Design by: Faisal Alvi

(For any suggested modifications please email: faisal.alvi@sse.habib.edu.pk)



Unit Outline

1. Introduction
2. C++
3. Anatomy of a C++ Program
4. Variables and Data Types
5. Input and Output
6. Loops
7. Functions
8. Summary



Introduction

- Object-oriented programming (OOP) is a programming paradigm based on the concept of objects, which can contain **data** and **code**:
 - **data** in the form of fields (often known as *attributes* or *properties*), and
 - **code** in the form of procedures or functions (often known as *methods*).
- In OOP, computer programs are designed by creating objects that interact with one another.
- Some of the most widely used programming languages using Object Oriented Programming are: C++, Java, Python (supports), etc.
- In this course we will be using C++ as the language of our choice for studying Object Oriented Programming.

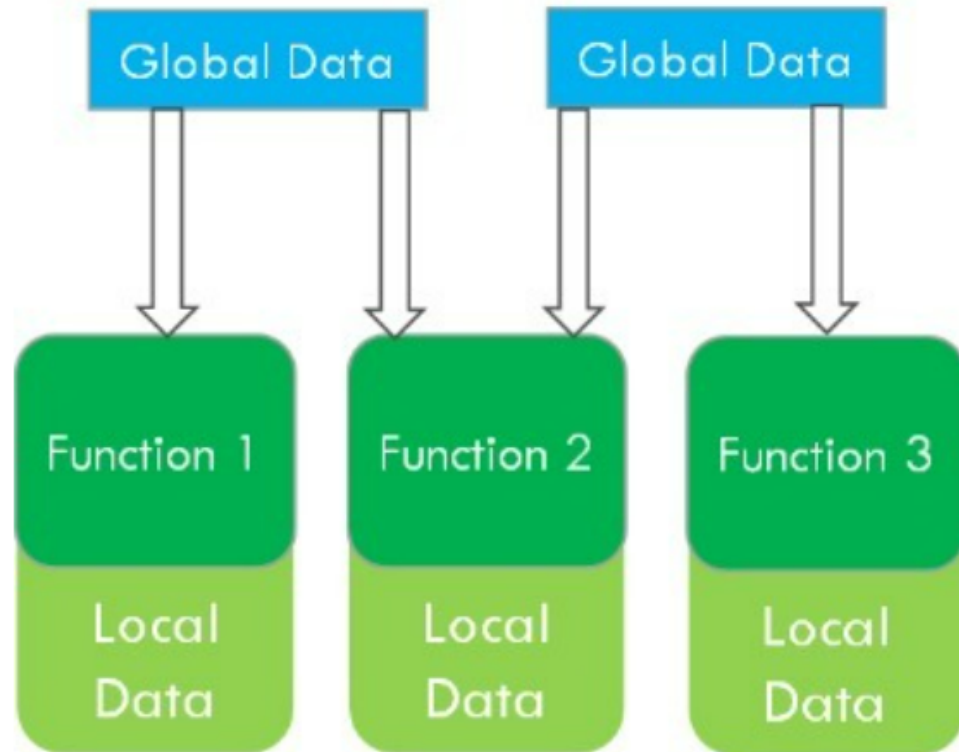


Why OOP Matters

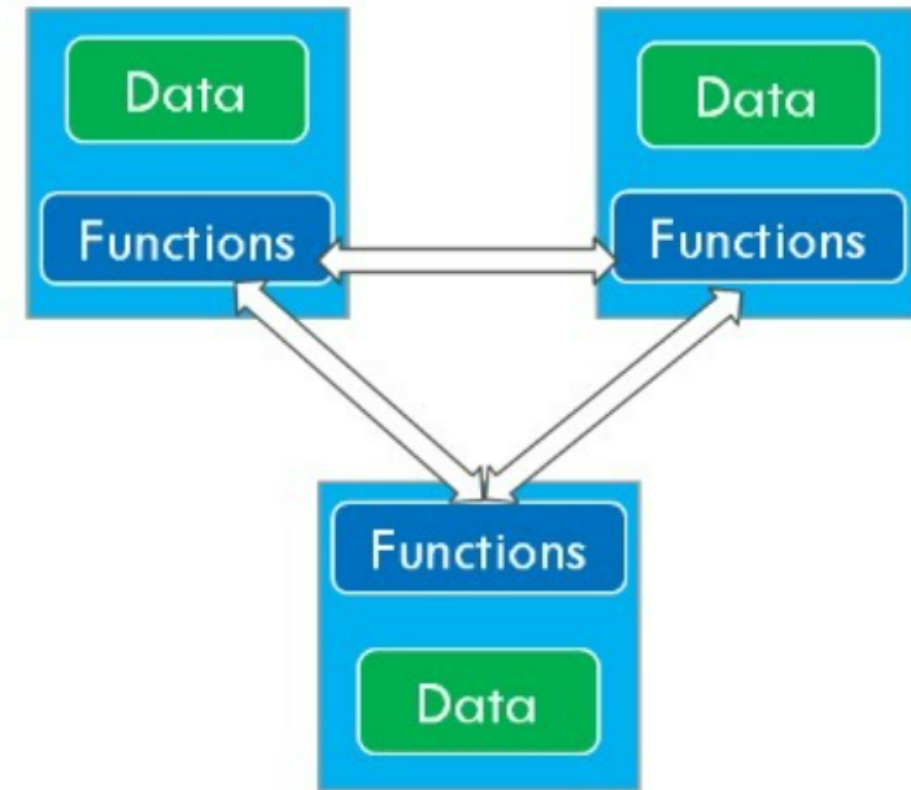
- Makes **complex software manageable** by breaking it into smaller, reusable parts.
- Encourages **reusability** through classes and inheritance.
- Supports **maintainability**—easier to update and expand programs.
- Industry relevance: Most modern applications (games, simulations, apps) are built with OOP principles.

Procedural VS Object Oriented Programming

Procedural Oriented Programming



Object Oriented Programming





Procedural VS Object Oriented Programming

```
#include <iostream>
using namespace std;

// Function that calculates area by taking width and height as inputs
double calculateArea(double width, double height) {
    return width * height;
}

int main() {
    double width = 5.0;
    double height = 3.0;

    // Call the function and print result
    double area = calculateArea(width, height);
    cout << "Area of Rectangle (Procedural): " << area << endl;

    return 0;
}
```

Don't Worry about syntax, we will study it!

Object Oriented Programming

```
#include <iostream>
using namespace std;

// Define a Rectangle class
class Rectangle {
private:
    double width;
    double height;

public:
    // Constructor
    Rectangle(double w, double h) {
        width = w;
        height = h;
    }

    // Method to calculate area
    double area() {
        return width * height;
    }
};

int main() {
    // Create an object of Rectangle
    Rectangle rect(5.0, 3.0);

    // Call the object's method
    cout << "Area of Rectangle (OOP): " << rect.area() << endl;

    return 0;
}
```

C++

- C++ is a mid-level, general-purpose programming language created by Danish computer scientist Bjarne Stroustrup.
- Extension of C, its earlier precursor.
- Supports both procedural and object oriented programming paradigms.
- Several standards; for this course, we use C++ 17.

C++ standards

Year	ISO/IEC Standard	Informal name
1998	14882:1998 ^[34]	C++98
2003	14882:2003 ^[35]	C++03
2011	14882:2011 ^[36]	C++11, C++0x
2014	14882:2014 ^[37]	C++14, C++1y
2017	14882:2017 ^[38]	C++17, C++1z
2020	14882:2020 ^[17]	C++20, C++2a
<u>TBA</u>	14882:2024	C++23, C++2b
<u>TBA</u>		C++26, C++2c

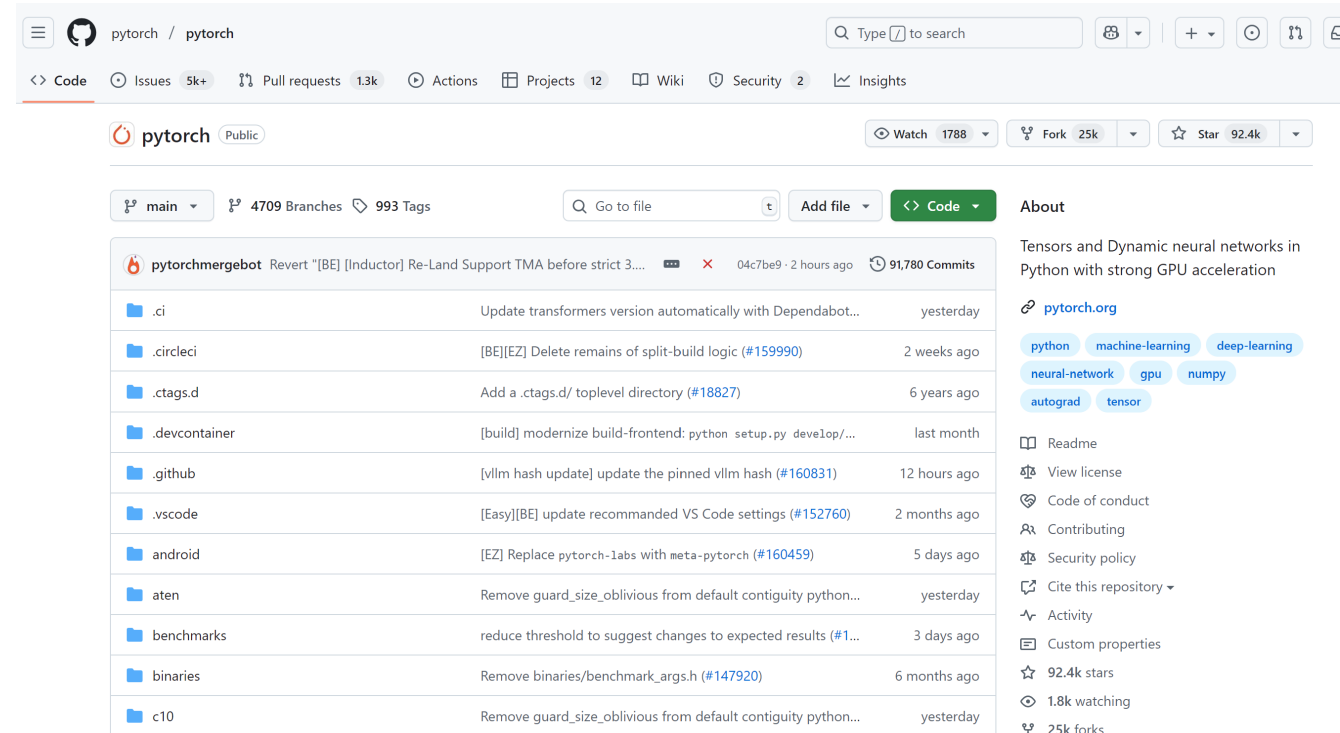


Why C++ Matters vs Python

- **Low-level control** → Direct access to memory (pointers, allocation, optimization).
- **Performance** → C++ is compiled and much faster; Python relies on it for speed.
- **Under the hood of Python** → PyTorch, TensorFlow, and many ML libraries are written in C++/CUDA.
- **Strong foundations** → Teaches how memory, data structures, and computation actually work.
- **Career relevance** → Robotics, game engines, embedded systems, and automotive software all rely on C++.
- **Big picture** → Python = productivity, C++ = power behind the scenes.

Why C++ Matters vs Python

- One of the most popular deep learning library i.e. Pytorch which is written in Python calls C++ under the hood



Why C++ Matters vs Python

- Pytorch (Python Friendly)

```
import torch

# Create two random tensors
a = torch.rand({1000, 1000})
b = torch.rand({1000, 1000})

# Matrix multiplication
c = torch.matmul(a, b)

print(c.size())
```

- C++ (under the hood)

```
#include <torch/torch.h>
#include <iostream>

int main() {
    // Create two random tensors
    torch::Tensor a = torch::rand({1000, 1000});
    torch::Tensor b = torch::rand({1000, 1000});

    // Matrix multiplication
    torch::Tensor c = torch::matmul(a, b);

    std::cout << c.size() << std::endl;
    return 0;
}
```



Syllabus!!



The C++ compiler : g++

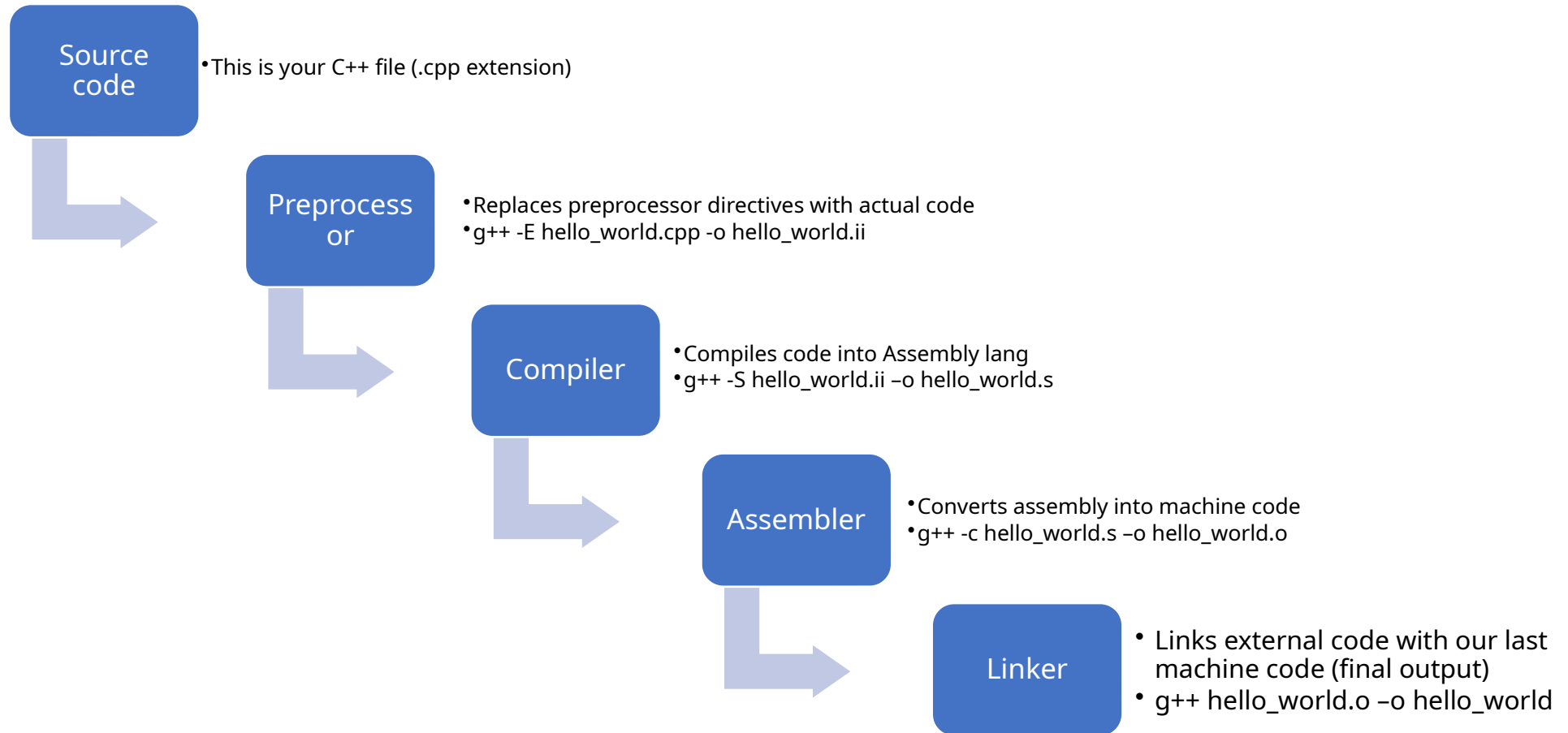
Preprocessor.....Compiler...Assembler...Linker
GNU tool chain



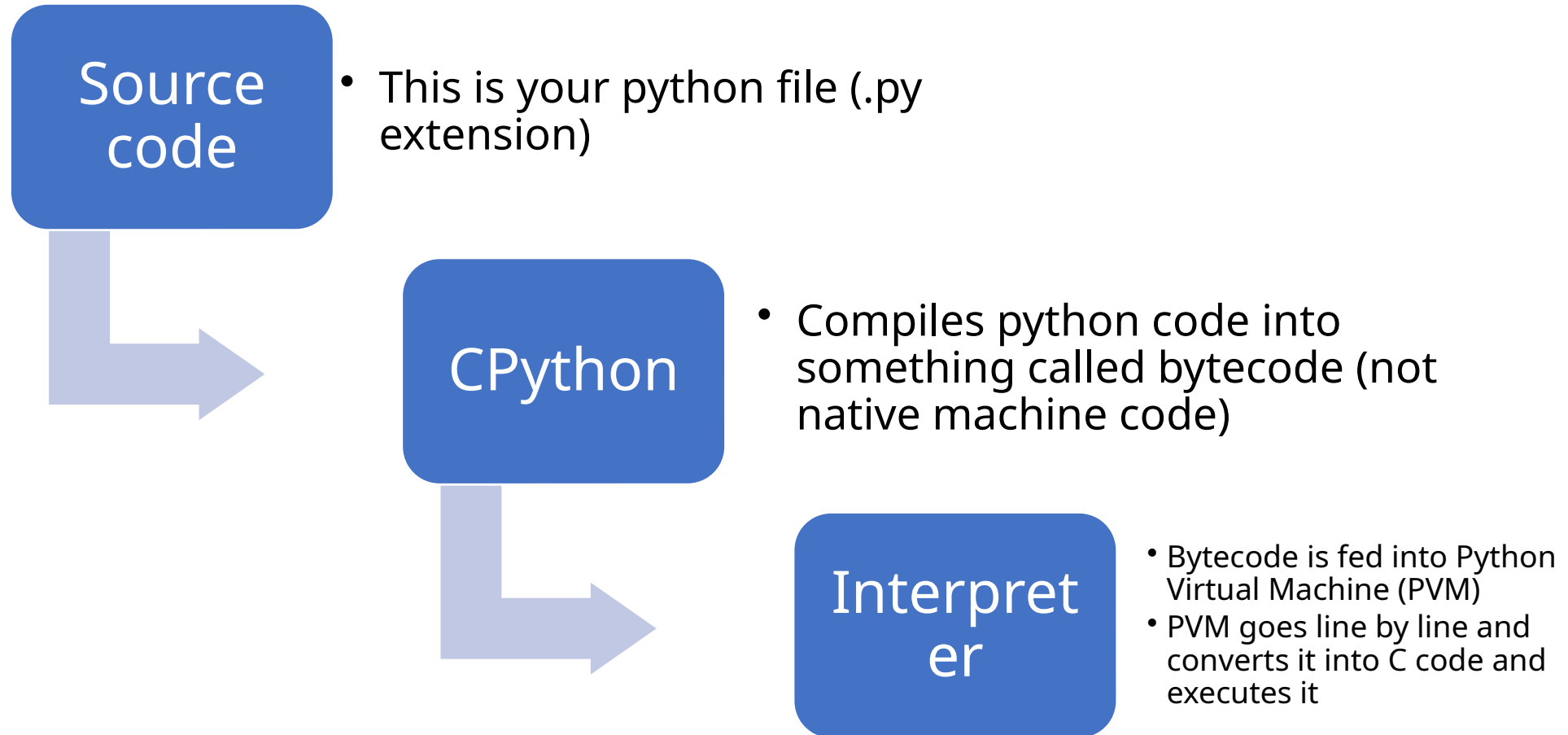
A short story about GNU

- GNU stands for “GNU Not Unix”... did you get the joke?
- It's a project started in 1983 by Richard Stallman set out to build a unix-like operating system which is opensource and free.
- Since then it has incorporated a lot of other software.
- One of them is g++, short for GNU C++.
- Its not just a single tool, it is a connection – a chain – of multiple tools that connect to each other
- So it is also called GNU tool chain

The GNU tool chain



How is this different from Python?





Anatomy of a C++ Program

```
1  #include <iostream>
```

Preprocessor Directive; Include files

```
2
```

```
3  using namespace std;
```

Using namespace

```
4
```

```
5  int main(void) {
```

Main Function; Observe the braces

```
6      cout << "Hello World" << endl;
```

```
7      return 0;
```

```
8  }
```

cout with the insertion operator



Anatomy of a C++ Program (contd.)

- Preprocessor Directive: `#include <iostream>`,
- instructs the preprocessor to include a section of standard C++ code, known as header `iostream`, that allows to perform standard input and output operations, such as writing the output of this program (Hello World) to the screen.
- In effect, the `#include` directive is replaced by the contents of the file indicated.



Anatomy of C++ Program (contd.)

- `using` Directive:
- C++ program can be divided into different namespaces. A namespace defines the scope of certain names; outside of the namespace they're unknown.
- The directive
- `using namespace std;`
- says that all the program statements that follow are within the `std` namespace.
- Various program components such as `cout` are declared within this namespace.



Anatomy of a C++ Program (contd.)

int main(**void**)

- The function named main is a special function in all C++ programs; it is the function called when the program is run.
- The execution of all C++ programs begins with the main function, regardless of where the function is actually located within the code.
- **void** indicates the parameter type of the function.
- **int** indicates the return type of the function.

Variables and Data Types

- A **variable** is a memory address where data can be **stored** and **changed**.
- Declaring a variable means specifying both its **name** and its **data type**.
- In C++ there are several data types – each specifying the type of the

TABLE 2.2 Basic C++ Variable Types

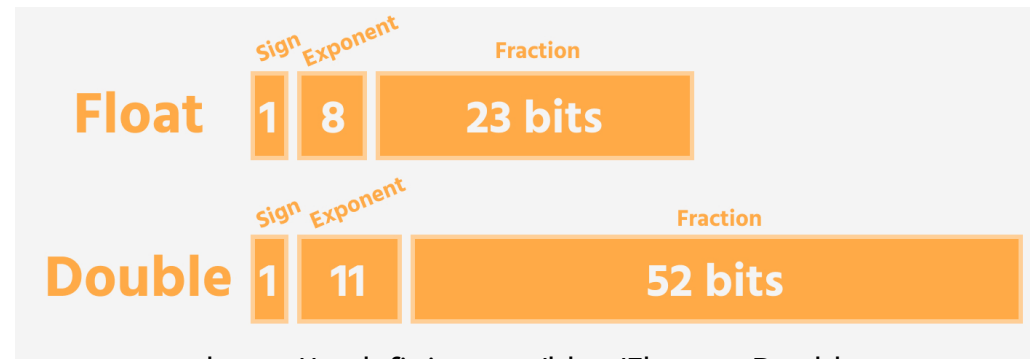
<i>Keyword</i>	<i>Numerical Range</i>		<i>Digits of Precision</i>	<i>Bytes of Memory</i>
	<i>Low</i>	<i>High</i>		
bool	false	true	n/a	1
char	−128	127	n/a	1
short	−32,768	32,767	n/a	2



Variables and Data Types

TABLE 2.2 Continued

<i>Keyword</i>	<i>Numerical Range</i>		<i>Digits of Precision</i>	<i>Bytes of Memory</i>
	<i>Low</i>	<i>High</i>		
int	−2,147,483,648	2,147,483,647	n/a	4
long	−2,147,483,648	2,147,483,647	n/a	4
float	3.4×10^{-38}	3.4×10^{38}	7	4
double	1.7×10^{-308}	1.7×10^{308}	15	8





Variables and Data Types

TABLE 2.3 Unsigned Integer Types

<i>Keyword</i>	<i>Numerical Range</i>		<i>Bytes of Memory</i>
	<i>Low</i>	<i>High</i>	
unsigned char	0	255	1
unsigned short	0	65,535	2
unsigned int	0	4,294,967,295	4
unsigned long	0	4,294,967,295	4



Primitive Data Types in C++

- **Integral Types**

- represent whole numbers and their negatives
- declared as int, short, or long

- **Character Types**

- represent single characters
- declared as char
- Stored by ASCII values

- **Boolean Type**

- declared as bool
- has only 2 values true/false
- will not print out directly

- **Floating Types**

- represent real numbers with a decimal point
- declared as float, or double
- Scientific notation where e (or E) stand for "times 10 to the " (.55-e6)



Variable Declaration

- A declaration tells the compiler to **allocate enough memory** to hold a value of this data type, and to **associate the identifier** with this location.
- Example:
`int count = 7;`
`float avgWeight = 155.5F;`



Type Conversions

- Handled automatically

```
int count = 7;  
float avgWeight = 155.5F;  
double totalWeight = count * avgWeight;
```

- When two operands of different types are encountered in the same expression, the lower-type variable is converted to the type of the higher-type variable.
- Order of types from highest to lowest:
 - long double > double > float > long > int > short > char
- Note: Remember the sizing table?



Type Conversions (Casting)

- Casting - forcing conversion by putting (type) in front of variable or expression. Used to insure that result is of desired type.
- Example: If you want to divide two integers and get a real result you must cast one to double so that a real divide occurs and store the result in a double.
- `int x = 5, y = 2; double z; z = static_cast <double> (x)/y; // 2.5`
- `int x = 5, y = 2; double z; z = (double) x/y; // 2.5`
- `int x = 5, y = 2; double z; z = static_cast <double> (x/y); // 2.0`
- `static_cast<int> (z)` - will truncate z, example `z= 2.9`, output = 2
- `static_cast <int> (x + 0.5)` - will round positive x {use () to cast complete expression}, example `x = 2.9`, output = 3.0 where we applied `x=int(2.9+0.5)`



Input/Output

- The identifier `cout` (pronounced “C out”) is actually an object. It is predefined in C++ to correspond to the standard output stream.
- A stream is an abstraction that refers to a flow of data.
- The standard output stream normally flows to the screen display—although it can be redirected to other output devices.
- The operator `<<` is called the **insertion** or **put to** operator. It directs the contents of the variable on its right to the object on its left.
- Example: Refer to the “HelloWorld” Program.



Input/Output

- The keyword `cin` (pronounced “C in”) is an object, predefined in C++ to correspond to the standard input stream.
- This stream represents data coming from the keyboard (unless it has been redirected).
- The `>>` is the **extraction** or **get from** operator.
- It takes the value from the stream object on its left and places it in the variable on its right.



Arithmetic Operators

- Operators: +, -, * /
- For floating numbers, the result is the same as Math operations.
- Note on integer division: the result is an integer. $7/2$ is 3.
- % (remainder or modulo) is the special operator just for integer. It yields an integer as the result. $7\%2$ is 1.
- Both / and % can only be used for positive integers.
- Precedence rule is similar to Math.



Relational Operators

- Expression that yields bool result
- Include:
- 6 Relational Operators

< <= > >= == !=

- 3 Logical Operators

! && ||

Decisions

- if-else statements
- Similar to Python with some syntactic differences:

```
# Compare and find the maximum value
if a >= b and a >= c:
    max_value = a
elif b >= a and b >= c:
    max_value = b
else:
    max_value = c
```

```
// Compare and find the maximum value
if (a >= b && a >= c) {
    max = a;
} else if (b >= a && b >= c) {
    max = b;
} else {
    max = c;
}
```



Loops

- while, do-while and for-loops
- Similar to Python with some syntactic differences (e.g. for-loop)

```
for i in range(1, 11):  
    print(i, end=" ")
```

```
// Loop from 1 to 10  
for (int i = 1; i <= 10; ++i) {  
    std::cout << i << " ";  
}
```

- Question to Explore: What is ++i? Is it different to i++?



Functions

- Functions in C are similar to those in Python with one major difference.
- Every function must have a return type, and each parameter (if present) must have a data type.
- For example in Python we can define a function increment which increments the variable

```
def increment(x):  
    return x + 1
```

- In C++, this will be: `int increment(int x)`

```
int increment(int x) {  
    return x + 1;  
}
```



Summary

- In this session we covered the basics of C++ Programming.
- Specifically we covered data types, type conversions, Arithmetic and Relational Expressions, conditional operators and loops.
- In the subsequent sessions, we will use these to build programs.

Class Exercise: Write a program in C++ that reads in the coefficients a , b and c of a quadratic equation $ax^2 + bx + c = 0$ and finds and prints the roots of the equation.

Bonus: Write the roots of equations that are complex in the proper form.